

Ship of Theseus Migration Plan: seqwish C++ → Rust

Executive Summary

Feasibility: HIGH ✓

This migration is **highly feasible** using a Ship of Theseus approach. The codebase has: - Clean layered architecture with natural boundaries - ~4,000 LOC total (manageable size) - Well-defined module interfaces - Existing test infrastructure - No global state/singleton

Strategy: FFI-based incremental replacement where Rust gradually replaces C++ components while maintaining a working binary at each step.

Migration Strategy: The Three Bridges

Bridge 1: C++ calls Rust (via extern "C")

Steps 1-8: Rust components expose C-compatible APIs, C++ gradually adopts them

Bridge 2: Dual Implementation (C++ & Rust coexist)

Steps 9-12: Core algorithm implemented in both, validated against each other

Bridge 3: Rust calls C++ (via bindgen/cxx)

Steps 13-15: Rust becomes primary, C++ components wrapped until replaced

15-Step Migration Path

Each step is: - ✓ Independently testable - ✓ Preserves full functionality - ✓ Reversible (can roll back) - ✓ Buildable (mixed C++/Rust builds)

PHASE 1: Foundation (Steps 1-3)

Goal: Establish Rust toolchain and prove FFI works

STEP 1: Project Setup & Hello World FFI Effort: 0.5 days Risk: Low

Actions:

```
# Create Rust library alongside C++
cargo init --lib seqwish-rs
```

File Changes: - Add Cargo.toml with crate-type = ["staticlib", "cdylib"] - Add build.rs for C header generation - Modify CMakeLists.txt to link Rust static library

Test Case:

```
// src/lib.rs
#[no_mangle]
pub extern "C" fn seqwish_rust_version() -> *const c_char {
```

```

    "0.1.0-rust\0".as_ptr() as *const c_char
}

// src/main.cpp (add)
extern "C" const char* seqwish_rust_version();
std::cerr << "Rust component: " << seqwish_rust_version() << std::endl;

```

Success Criteria: - ✓ Mixed build completes - ✓ Binary runs and prints Rust version - ✓ All existing tests pass

STEP 2: Migrate tempfile.hpp **→ Rust** **Effort:** 1 day **Risk:** Low **Files:** src/tempfile.hpp (34 LOC)

Why First? - Zero seqwish dependencies - Simple RAI wrapper - Used throughout codebase (validates FFI integration)

Rust Implementation:

```

// seqwish-rs/src/tempfile.rs
use std::path::PathBuf;
use std::fs;

pub struct TempFile {
    path: PathBuf,
}

impl TempFile {
    pub fn new(prefix: &str) -> std::io::Result<Self> {
        let path = std::env::temp_dir().join(format!("_XXXXXX", prefix));
        // Use mkstemp equivalent
        Ok(TempFile { path })
    }
}

impl Drop for TempFile {
    fn drop(&mut self) {
        let _ = fs::remove_file(&self.path);
    }
}

// C FFI wrapper
#[repr(C)]
pub struct CTempFile {
    inner: *mut TempFile,
}

#[no_mangle]
pub extern "C" fn tempfile_new(prefix: *const c_char) -> CTempFile {
    // ... implementation
}

#[no_mangle]
pub extern "C" fn tempfile_path(tf: CTempFile) -> *const c_char {
    // ... implementation
}

```

```

}

#[no_mangle]
pub extern "C" fn tempfile_free(tf: CTempFile) {
    // ... implementation
}

```

C++ Wrapper (keep interface identical):

```

// src/tempfile.hpp (modified to call Rust)
#include "seqwish_rs.h"

class temp_file {
    CTempFile handle;
public:
    temp_file(const std::string& prefix) {
        handle = tempfile_new(prefix.c_str());
    }
    ~temp_file() { tempfile_free(handle); }
    std::string get_name() const {
        return tempfile_path(handle);
    }
};

```

Test Case: - Modify existing code that uses temp_file - Verify files are created/deleted properly
 - Run full test suite (should pass unchanged)

Success Criteria: - ✓ All callsites work without modification - ✓ No memory leaks (valgrind clean) - ✓ File cleanup verified

STEP 3: Migrate position.hpp → **Rust** **Effort:** 1 day **Risk:** Low **Files:** src/position.hpp (121 LOC)

Why Next? - Zero dependencies - Pure data types (no I/O) - Used everywhere (validates data marshaling)

Rust Implementation:

```

// seqwish-rs/src/position.rs
#[repr(C)]
#[derive(Debug, Copy, Clone, PartialEq, Eq)]
pub struct Pos {
    offset: u64,
    is_rev: bool,
}

impl Pos {
    pub fn encode(offset: u64, is_rev: bool) -> u64 {
        (offset << 1) | (is_rev as u64)
    }

    pub fn decode(encoded: u64) -> Self {
        Pos {
            offset: encoded >> 1,
            is_rev: (encoded & 1) != 0,
        }
    }
}

```

```

    }
}

// No FFI needed - can use #[repr(C)] directly in C++

```

C++ Integration:

```

// src/position.hpp (simplified, delegates to Rust)
#include "seqwish_rs.h"
using pos_t = uint64_t;

inline pos_t make_pos_t(size_t offset, bool is_rev) {
    return pos_encode(offset, is_rev);
}

```

Test Case:

```

#[test]
fn test_position_encoding() {
    assert_eq!(Pos::encode(100, false), 200);
    assert_eq!(Pos::encode(100, true), 201);
    let p = Pos::decode(201);
    assert_eq!(p.offset, 100);
    assert_eq!(p.is_rev, true);
}

```

Success Criteria: - ✓ Rust unit tests pass - ✓ C++ integration tests pass - ✓ Binary compatibility verified (same encoded values)

PHASE 2: Utilities (Steps 4-6)

Goal: Replace standalone utility modules

STEP 4: Migrate dna.hpp → Rust **Effort:** 1.5 days **Risk:** Low **Files:** src/dna.hpp (39 LOC)

Why? - No dependencies - Simple encoding (A/C/G/T → 0/1/2/3) - Tests string handling over FFI

Rust Implementation:

```

// seqwish-rs/src/dna.rs
pub fn encode(c: u8) -> u8 {
    match c.to_ascii_uppercase() {
        b'A' => 0,
        b'C' => 1,
        b'G' => 2,
        b'T' => 3,
        _ => 0, // N or invalid
    }
}

pub fn decode(e: u8) -> u8 {
    match e {
        0 => b'A',
        1 => b'C',
        2 => b'G',
    }
}

```

```

        3 => b'T',
        _ => b'N',
    }
}

pub fn reverse_complement(seq: &[u8]) -> Vec<u8> {
    seq.iter().rev().map(|&c| {
        match c {
            b'A' => b'T',
            b'T' => b'A',
            b'C' => b'G',
            b'G' => b'C',
            _ => c,
        }
    }).collect()
}

// C FFI
#[no_mangle]
pub extern "C" fn dna_reverse_complement(
    seq: *const c_char,
    len: usize,
    out: *mut c_char
) {
    // ... implementation
}

```

Test Case: - Verify `reverse_complement("ACGT") == "ACGT"` - Test with existing sequence data from test files

Success Criteria: - ✓ Rust unit tests pass - ✓ seqwish output unchanged (bit-for-bit)

STEP 5: Migrate `cigar.hpp` → **Rust** **Effort:** 2 days **Risk:** Low-Medium **Files:** `src/cigar.hpp` (189 LOC)

Why? - No dependencies - Critical for PAF parsing (validates complex logic)

Rust Implementation:

```

// seqwish-rs/src/cigar.rs
#[derive(Debug, Clone)]
pub enum CigarOp {
    Match(u32),
    Insert(u32),
    Delete(u32),
    Equal(u32),
    Diff(u32),
    // ... etc
}

pub fn parse_cigar(s: &str) -> Vec<CigarOp> {
    let mut ops = Vec::new();
    let mut num = 0u32;
}

```

```

for c in s.bytes() {
    if c.is_ascii_digit() {
        num = num * 10 + (c - b'0') as u32;
    } else {
        ops.push(match c {
            b'M' => CigarOp::Match(num),
            b'I' => CigarOp::Insert(num),
            b'D' => CigarOp::Delete(num),
            // ... etc
        });
        num = 0;
    }
}
ops
}

```

Test Case: - Parse all CIGAR strings from test PAF files - Verify `to_length()` calculations match C++

Success Criteria: - ✓ Identical parsing results - ✓ Performance within 10% of C++

STEP 6: Migrate `mmap.hpp` → **Rust** **Effort:** 2 days **Risk:** Medium **Files:** `src/mmap.hpp` (88 LOC)

Why? - Critical infrastructure (used everywhere) - Tests memory-mapped I/O over FFI

Rust Implementation:

```

// seqwish-rs/src/mmap.rs
use memmap2::MmapMut;

pub struct MmapFile {
    mmap: MmapMut,
}

impl MmapFile {
    pub fn new(path: &str, size: usize) -> std::io::Result<Self> {
        let file = OpenOptions::new()
            .read(true)
            .write(true)
            .create(true)
            .open(path)?;
        file.set_len(size as u64)?;
        let mmap = unsafe { MmapMut::map_mut(&file)? };
        Ok(MmapFile { mmap })
    }

    pub fn as_ptr(&self) -> *const u8 {
        self.mmap.as_ptr()
    }
}

// C FFI
#[no_mangle]

```

```
pub extern "C" fn mmap_open(path: *const c_char, size: usize) -> *mut MmapFile {
    // ... implementation
}
```

Test Case: - Create mmapped files from both C++ and Rust - Verify data consistency

Success Criteria: - ✓ No crashes or corruption - ✓ Works on Linux/macOS

PHASE 3: Data Structures (Steps 7-9)

Goal: Replace memory-mapped containers

STEP 7: Migrate `dmultimap.hpp` → **Rust** **Effort:** 4 days **Risk:** Medium-High **Files:** `src/dmultimap.hpp` (wrapper around `mmulti`)

Why? - Core data structure (used in 4+ algorithms) - Tests Rust's memory model for complex structures

Rust Implementation:

```
// seqwish-rs/src/dmultimap.rs
use mmap_interval_tree::IntervalTree; // Or custom impl

pub struct DMultiMap<K, V> {
    backing_file: PathBuf,
    tree: IntervalTree<K, V>,
}

impl<K: Ord, V> DMultiMap<K, V> {
    pub fn open(path: &Path) -> std::io::Result<Self> {
        // Memory-map file
    }

    pub fn append(&mut self, key: K, value: V) {
        // Append to interval tree
    }

    pub fn get(&self, key: &K) -> impl Iterator<Item = &V> {
        // Return iterator over values
    }
}
```

Challenge: `mmulti` has no Rust equivalent **Options:** 1. Port `mmulti` to Rust (3-4 days) 2. Use existing crate like `interval-tree` (may need `mmap` integration) 3. Keep `mmulti` in C++, wrap with Rust FFI (temporary)

Recommendation: Option 3 initially, then Option 1

Test Case: - Migrate one algorithm (e.g., `seqindex`) to use Rust `dmultimap` - Verify identical output

Success Criteria: - ✓ Memory usage identical - ✓ Performance within 20% (acceptable for first pass) - ✓ All queries return same results

STEP 8: Migrate seqindex.hpp → **Rust** **Effort:** 3 days **Risk:** Medium **Files:** src/seqindex.hpp (242 LOC)

Why? - Well-isolated module - Uses dmultimap (validates Step 7) - Prepares for algorithm migration

Rust Implementation:

```
// seqwish-rs/src/seqindex.rs
use crate::dmultimap::DMultiMap;

pub struct SeqIndex {
    seqs: Vec<String>,
    index: DMultiMap<String, usize>, // name -> seq_id
}

impl SeqIndex {
    pub fn from_fasta(path: &Path) -> std::io::Result<Self> {
        // Parse FASTA
    }

    pub fn get_seq(&self, name: &str) -> Option<&str> {
        // Lookup
    }
}

// C FFI wrapper
#[repr(C)]
pub struct CSeqIndex {
    inner: *mut SeqIndex,
}
```

Test Case: - Index test FASTA files - Compare lookup results with C++

Success Criteria: - ✓ Identical indexing - ✓ Memory usage similar

STEP 9: Migrate PAF Parsing (paf.hpp, alignments.cpp) → **Rust** **Effort:** 3 days **Risk:** Medium **Files:** src/paf.hpp (74 LOC), src/alignments.cpp (108 LOC)

Why? - Input layer (validates I/O intensive code) - Used by main pipeline

Rust Implementation:

```
// seqwish-rs/src/paf.rs
use std::io::{BufRead, BufReader};

#[derive(Debug)]
pub struct PafRecord {
    pub query_name: String,
    pub query_len: usize,
    pub query_start: usize,
    pub query_end: usize,
    pub strand: char,
    pub target_name: String,
    // ... 12 standard fields
    pub cigar: Option<String>,
}
```



```

}

pub fn parse_paf<R: BufRead>(reader: R) -> impl Iterator<Item = PafRecord> {
    reader.lines().filter_map(|line| {
        let line = line.ok()?;
        let fields: Vec<&str> = line.split('\t').collect();
        if fields.len() < 12 { return None; }

        Some(PafRecord {
            query_name: fields[0].to_string(),
            query_len: fields[1].parse().ok()?,
            // ... parse all fields
        })
    })
}

```

Test Case: - Parse test PAF files - Compare record counts and values with C++

Success Criteria: - ✓ Identical parsing - ✓ Performance within 10%

PHASE 4: Core Algorithms (Steps 10-12)

Goal: Migrate the 6-phase pipeline (HIGHEST RISK)

STEP 10: Dual Implementation - compact.cpp (**Rust + C++**) **Effort:** 5 days **Risk:** High
Files: src/compact.cpp (204 LOC)

Why This Order? - compact is simpler than transclosure (good warmup) - Tests graph algorithms in Rust

Strategy: Run **both** implementations, compare outputs

Rust Implementation:

```

// seqwish-rs/src/compact.rs
use crate::graph::{Graph, NodeId};

pub fn compact_nodes(graph: &mut Graph) {
    // Implement node compaction algorithm
    // Remove singleton nodes, merge linear chains
}

// C FFI
#[no_mangle]
pub extern "C" fn seqwish_compact_rust(graph_ptr: *mut CGraph) {
    // ... implementation
}

```

Test Case:

```

// src/main.cpp (add validation)
auto graph_cpp = graph; // Copy
auto graph_rust = graph; // Copy

compact_nodes(graph_cpp); // C++ version
seqwish_compact_rust(&graph_rust); // Rust version

```

```
assert(graph_cpp == graph_rust); // Compare node counts, edges
```

Success Criteria: - ✓ Both produce identical graphs - ✓ Rust version within 30% performance (optimization comes later)

STEP 11: Dual Implementation - transclosure.cpp (Rust + C++) **Effort:** 8 days **Risk:** VERY HIGH **Files:** src/transclosure.cpp (733 LOC - most complex file)

Why Scary? - Lock-free parallel algorithm - Uses atomic operations extensively - Critical for correctness (transitive closure of overlaps)

Strategy: Incremental port with extensive validation

Rust Implementation:

```
// seqwish-rs/src/transclosure.rs
use std::sync::atomic::{AtomicU64, Ordering};
use rayon::prelude::*;

pub fn compute_transitive_closure(
    seqs: &[Sequence],
    alignments: &[Alignment],
    threads: usize
) -> ClosureGraph {
    // Phase 1: Build initial overlap graph
    let overlap_graph = build_overlap_graph(alignments);

    // Phase 2: Compute transitive closure (parallel)
    let closure = overlap_graph.par_iter()
        .map(|node| compute_node_closure(node))
        .collect();

    closure
}
```

Test Case:

```
# Run both versions on small dataset
seqwish_cpp test.paf > output_cpp.gfa
seqwish_rust test.paf > output_rust.gfa
diff output_cpp.gfa output_rust.gfa
```

```
# Validate graph properties
python scripts/validate_graph.py output_cpp.gfa output_rust.gfa
```

Success Criteria: - ✓ **Identical GFA output** (bit-for-bit if possible) - ✓ Graph isomorphism verified - ✓ Performance within 50% (parallelism is hard)

Fallback Plan: If Rust version differs, keep C++ as reference implementation

STEP 12: Migrate Remaining Algorithms (links.cpp) **Effort:** 4 days **Risk:** Medium **Files:** src/links.cpp (213 LOC)

Why Last? - Depends on transclosure output - Simpler than transclosure

Rust Implementation:

```
// seqwish-rs/src/links.rs
pub fn derive_links(graph: &Graph) -> Vec<Link> {
    // Derive links from compacted nodes
}
```

Test Case: - Compare link counts with C++ - Verify GFA output

Success Criteria: - ✓ Identical link derivation

PHASE 5: I/O and Main (Steps 13-14)

Goal: Replace main entry point

STEP 13: Migrate GFA Output (gfa.cpp, vgp.cpp) → Rust **Effort:** 3 days **Risk:** Low **Files:** src/gfa.cpp (77 LOC), src/vgp.cpp (91 LOC)

Rust Implementation:

```
// seqwish-rs/src/gfa.rs
use std::io::Write;

pub fn write_gfa<W: Write>(graph: &Graph, writer: &mut W) -> std::io::Result<()> {
    // Write GFA format
    writeln!(writer, "H\tVN:Z:1.0");
    for node in graph.nodes() {
        writeln!(writer, "S\t{}\t{}", node.id, node.seq)?;
    }
    // ... write edges
    Ok(())
}
```

Test Case: - Generate GFA from same graph - diff output with C++

Success Criteria: - ✓ Byte-for-byte identical

STEP 14: Migrate Main Entry Point (main.cpp) → Rust **Effort:** 2 days **Risk:** Low **Files:** src/main.cpp (384 LOC - mostly arg parsing)

Rust Implementation:

```
// seqwish-rs/src/main.rs
use clap::Parser;

#[derive(Parser)]
struct Args {
    #[arg(short, long)]
    paf_file: PathBuf,

    #[arg(short, long)]
    threads: Option<usize>,

    // ... other args
}
```

```

}

fn main() -> Result<()> {
    let args = Args::parse();

    // Call Rust implementation
    let alignments = parse_paf(&args.paf_file)?;
    let graph = compute_transitive_closure(&alignments, args.threads)?;
    let compacted = compact_nodes(graph);
    write_gfa(&compacted, &mut std::io::stdout())?;

    Ok(())
}

```

Test Case: - Run full pipeline on test data - Compare with C++ version

Success Criteria: - ✓ CLI identical (drop-in replacement) - ✓ Output identical

STEP 15: Remove C++ Code (Ship Fully Replaced) Effort: 1 day **Risk:** Low

Actions: - Remove src/*.cpp, src/*.hpp - Update README.md, CMakeLists.txt → Cargo.toml - Update CI/CD

Success Criteria: - ✓ Pure Rust build - ✓ All tests pass - ✓ Documentation updated

Critical Success Factors

1. Testing Strategy

Level 1: Unit Tests

```

// Every module has Rust unit tests
#[cfg(test)]
mod tests {
    #[test]
    fn test_position_encoding() {
        assert_eq!(encode(100, false), 200);
    }
}

```

Level 2: Integration Tests

```

# Run both C++ and Rust on same inputs
./test_parity.sh test_data/small.paf
# Compares outputs, asserts identical

```

Level 3: Regression Tests

```

# Use existing test suite (test/test.sh)
# Must pass at every step
make test

```

Level 4: Property Tests

```

// Use quickcheck/proptest for fuzzing
#[quickcheck]

```

```
fn prop_reverse_complement_involution(seq: Vec<u8>) -> bool {
    let rc = reverse_complement(&seq);
    let rcrc = reverse_complement(&rc);
    seq == rcrc
}
```

2. Performance Validation

Benchmarks:

Add benchmarks at each step
cargo bench

Compare with C++ baseline
hyperfine './seqwish_cpp test.paf' './seqwish_rust test.paf'

Acceptable Performance Loss: - Steps 1-9 (utilities): 0-20% slower OK - Steps 10-12 (algorithms): 0-50% slower OK initially - Step 15 (final): Must match C++ within 10%

Optimization Phase: After Step 15, dedicate 1-2 weeks to optimization

3. Documentation

At Each Step:

MIGRATION_LOG.md
Step 3: position.hpp → Rust
- Date: 2025-11-04
- Status: Complete
- Performance: Rust 5% faster (zero-cost abstraction)
- Issues: None
- Rollback: N/A

4. Rollback Plan

Each step is reversible:

git checkout step-2-tempfile *# Known good state*

5. Dependency Strategy

Critical Dependencies:

C++ Library	Rust Equivalent	Strategy
SDSL-lite	Custom impl	Port incrementally (Steps 7-12)
mmmulti	Custom impl	Port in Step 7
atomic_queue	crossbeam	Drop-in replacement
BBHash	Custom impl	Port in Step 10
args	clap	Drop-in replacement

SDSL Porting: - bit_vector → Use bitvec crate - int_vector → Custom implementation - wavelet_tree → Port from SDSL (2-3 days)

Risk Mitigation

High-Risk Areas

1. Transitive Closure (Step 11) - Risk: Most complex algorithm, lock-free parallelism - **Mitigation:** - Port incrementally (serialize first, parallelize later) - Extensive property testing - Keep C++ version as reference - Consider formal verification tools (Miri, LOOM)

2. Memory-Mapped Structures (Steps 6-7) - Risk: Rust's safety model vs. C++ raw pointers - **Mitigation:** - Use `unsafe` judiciously with safety comments - Encapsulate in safe APIs - Audit with `cargo-geiger`

3. Performance Regression - Risk: Rust may be slower initially - **Mitigation:** - Profile with `perf`, `flamegraph` - Use `#[inline]`, PGO, LTO - Benchmark continuously

Blockers

Potential Showstoppers: 1. SDSL has no Rust equivalent → **Mitigation:** Port key structures (4-5 days) 2. Lock-free algorithms differ → **Mitigation:** Use `crossbeam`, `parking_lot` 3. Memory-mapped I/O performance → **Mitigation:** Use `mmap2`, `mmap-rs`

Timeline Estimate

Phase	Steps	Effort	Calendar Time
1: Foundation	1-3	3 days	1 week
2: Utilities	4-6	5.5 days	1.5 weeks
3: Data Structures	7-9	10 days	2.5 weeks
4: Core Algorithms	10-12	17 days	4 weeks
5: I/O and Main	13-14	5 days	1.5 weeks
6: Finalization	15	1 day	0.5 weeks
Total	1-15	41.5 days	~11 weeks

Assumptions: - 1 developer, 4 hours/day focused work - Context refreshes every 3-5 days (accounted for) - 20% buffer for unknowns

Optimistic: 8 weeks **Realistic:** 11 weeks **Pessimistic:** 16 weeks (if SDSL port takes longer)

Context Refresh Strategy

Problem: Complex migration requires many context refreshes

Solution: Self-documenting checkpoints

At Each Step:

```
# Create checkpoint
git tag step-3-position
git push --tags

# Document state
cat > CHECKPOINT_3.md <<EOF
# Checkpoint 3: position.hpp migrated
```

```

## What's Done
- position.hpp → Rust (seqwish-rs/src/position.rs)
- Tests passing: cargo test position
- Integration: C++ calls Rust via #include "seqwish_rs.h"

## What's Next
- Step 4: dna.hpp → Rust

## How to Resume
1. Read RUST_MIGRATION_PLAN.md (this file)
2. Read CHECKPOINT_3.md (context)
3. Run: cargo test && make test
4. Continue to Step 4

## Current Issues
- None

## Performance
- Rust: 98 s, C++: 102 s (identical)
EOF

```

Resume Protocol:

```

# On context refresh
1. Read RUST_MIGRATION_PLAN.md (this file)
2. git tag --list | grep step # Find last checkpoint
3. Read CHECKPOINT_N.md # Get exact state
4. Run tests to verify state
5. Continue to next step

```

Success Metrics

Migration Complete When: - ✓ All C++ code removed - ✓ Pure Rust build (cargo build --release) - ✓ All tests pass (cargo test && ./test/test.sh) - ✓ Performance within 10% of C++ baseline - ✓ Memory usage similar - ✓ Output identical (GFA bit-for-bit)

Definition of “Testable at Each Step”: Each step must have: 1. Rust unit tests (cargo test) 2. C++/Rust integration test (both produce same output) 3. Full regression test passes (make test)

Alternatives Considered

Alternative 1: Full Rewrite (Rejected)

- **Pros:** Clean slate, idiomatic Rust
- **Cons:** High risk, long feedback loop, hard to test

Alternative 2: Gradual Type Migration (Rejected)

- **Pros:** Safer than full rewrite
- **Cons:** Requires mixed C++/Rust in same files (messy)

Alternative 3: Ship of Theseus (Selected)

- **Pros:** Testable at each step, reversible, incremental
 - **Cons:** Requires FFI overhead (temporary), longer timeline
-

Open Questions

1. **SDSL Porting:** Full port or minimal subset?
 - **Recommendation:** Minimal subset (bit_vector, int_vector only)
 2. **Parallel Strategy:** rayon vs. manual threading?
 - **Recommendation:** rayon for simplicity, optimize later
 3. **Memory Mapping:** mmap2 or custom?
 - **Recommendation:** mmap2 (well-tested)
 4. **FFI Overhead:** Acceptable?
 - **Recommendation:** Yes, removed by Step 15
-

Resources

Learning Rust (if needed): - The Rust Book: <https://doc.rust-lang.org/book/> - Rust by Example: <https://doc.rust-lang.org/rust-by-example/> - FFI Guide: <https://doc.rust-lang.org/nomicon/ffi.html>

Tools: - cargo-geiger: Audit unsafe code - cargo-flamegraph: Profiling - cargo-criterion: Benchmarking - miri: Detect undefined behavior

Similar Migrations: - ripgrep (grep replacement) - bat (cat replacement) - fd (find replacement)

Conclusion

This migration is HIGHLY FEASIBLE.

The seqwish codebase is: - Small (~4K LOC) - Well-structured (clean layers) - Well-tested (existing test suite) - Has clear module boundaries

The Ship of Theseus approach: - Minimizes risk (testable at each step) - Maintains working binary throughout - Survives context refreshes (checkpoint system) - Reversible (can roll back any step)

Recommendation: Proceed with Step 1 immediately.

Critical Path: Steps 1 → 2 → 3 → 7 → 10 → 11 → 14 → 15

Success Probability: 85% (high confidence)

Quick Start

```
# Step 1: Setup
cargo init --lib seqwish-rs
cd seqwish-rs
```

```
# Add to Cargo.toml
```



```

[lib]
crate-type = ["staticlib", "cdylib"]

# Create first FFI function
cat > src/lib.rs <<EOF
use std::ffi::c_char;

#[no_mangle]
pub extern "C" fn seqwish_rust_version() -> *const c_char {
    "0.1.0-rust\0".as_ptr() as *const c_char
}
EOF

# Build
cargo build --release

# Add to CMakeLists.txt
target_link_libraries(seqwish seqwish-rs/target/release/libseqwish_rs.a)

# Test
make && ./bin/seqwish --version

```

Ready to start? Begin with Step 1!